OWASP JUICE SHOP VULNERABILITY ASSESSMENT REPORT

PREPARED BY MARK SUSSACK

1. Introduction

This vulnerability assessment was carried out on the OWASP Juice Shop application to identify security weaknesses that could affect the confidentiality, integrity and availability of the application and its users.

The assessment focused mainly on testing the web application, its API endpoints, user input fields and exposed services. Different techniques were used, including browser Developer Tools, inspection of HTTP requests and responses, input testing, endpoint analysis and FTP enumeration.

The main purpose of the assessment was to understand how vulnerabilities can be identified in a web application and how such vulnerabilities can affect the security of the system.

2. Objective

The main objectives of the assessment were:

- To identify vulnerabilities existing in the OWASP Juice Shop application.
- To examine how the application handles user input.
- To identify insecure or exposed API endpoints.
- To check whether unauthorized information or files can be accessed.
- To identify security misconfigurations.
- To assess the possible impact of the identified vulnerabilities.
- To provide recommendations for improving the security of the application.

3. Scope

The assessment was limited to the OWASP Juice Shop application and the services exposed by the application in the testing environment.

The following areas were considered:

- Web application interface
- API endpoints
- Search functionality
- Product-related requests
- Browser Developer Tools
- FTP service

• Error handling
• Application security configuration
• User input handling

4. Methodology

The assessment was performed using a combination of manual testing and security testing techniques.

First, the application was accessed through a web browser and its functionality was explored. Developer Tools were then used to inspect network requests and identify endpoints used by the application.

After identifying interesting endpoints, the requests and parameters were examined to determine whether user-controlled input was properly validated.

The search functionality was tested for Cross-Site Scripting. API requests related to products were inspected for possible injection vulnerabilities.

In addition, FTP services were examined to determine whether files were exposed through the service. General application responses were also checked for security misconfiguration and improper error handling.

5. Tools Used

• Web Browser
• Browser Developer Tools
• Burp Suite
• FTP client / FTP enumeration tools
• Terminal
• OWASP Juice Shop

6. Findings Summary

No.	Vulnerability	Severity
1	Exposed API Endpoint / Information Disclosure	Medium
2	Reflected Cross-Site Scripting (XSS)	High
3	SQL Injection	Critical
4	FTP File Exposure	High
5	Security Misconfiguration	Medium
6	Improper Error Handling / Information Disclosure	Medium

7	Sensitive Information Exposure	Medium

7.1 Exposed API Endpoint / Information Disclosure

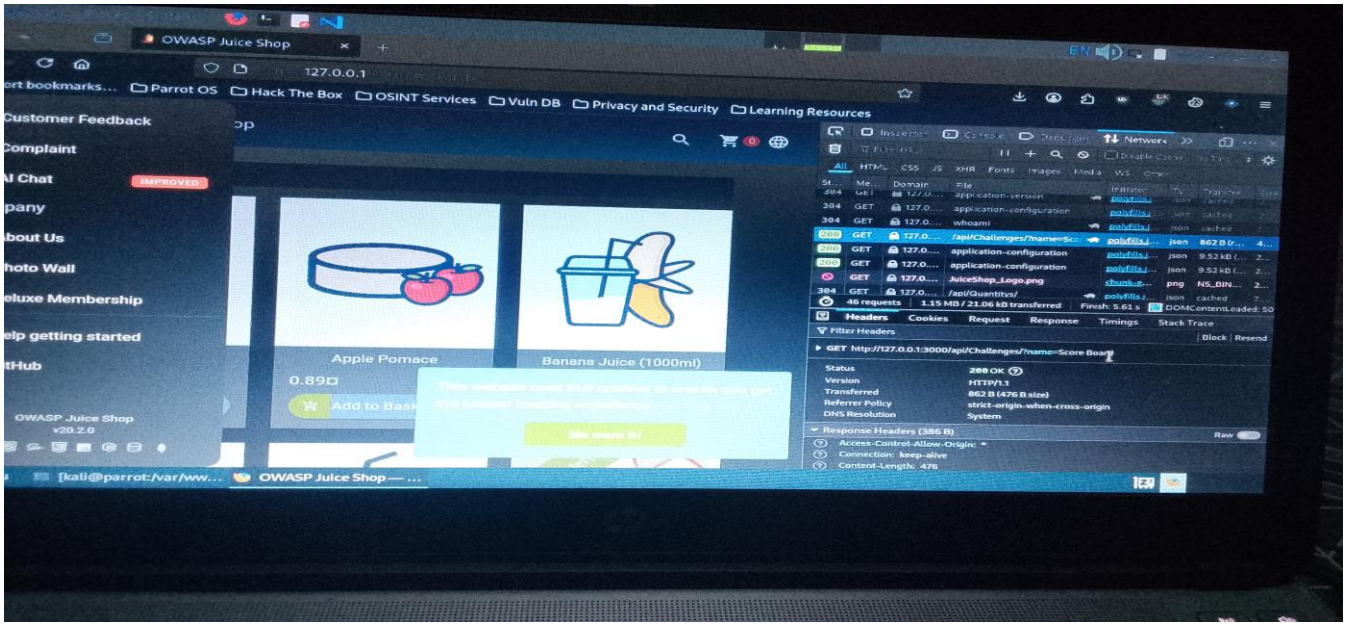
During the assessment, the browser Developer Tools were used to monitor the network traffic generated by the application.

While interacting with the application, API endpoints used by the frontend were identified. Some of these endpoints could be viewed directly from the browser's network requests.

This shows that the application exposes information about its backend API structure to the client.

Impact:
Exposed endpoints can help an attacker understand how the application communicates with its backend. If sensitive endpoints are not properly protected, an attacker may be able to interact with them directly without using the normal application interface.

Recommendation:
- Implement proper authentication and authorization on every sensitive endpoint.
- Avoid exposing unnecessary information in API responses.
- Review all publicly accessible endpoints.



Evidence:

7.2 Reflected Cross-Site Scripting (XSS)

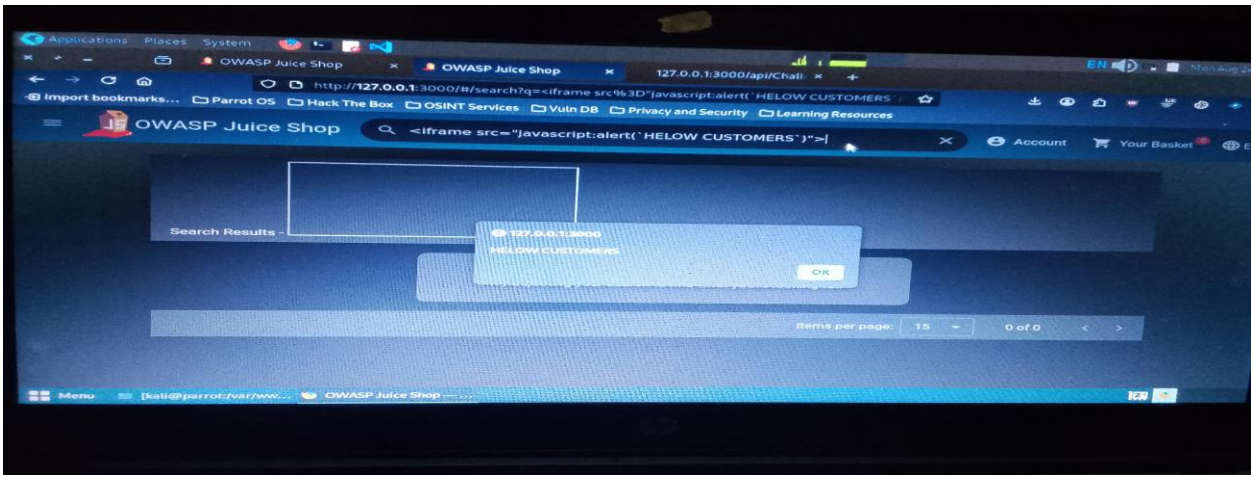
A reflected XSS vulnerability was identified in the application's search functionality.

The search parameter was tested with a JavaScript payload. The supplied input was reflected by the application without being properly encoded or sanitized, resulting in JavaScript execution in the browser.

Impact:
An attacker could potentially use the vulnerability to execute JavaScript in another user's browser. Depending on the application's security controls, this could lead to phishing, modification of page content or unauthorized actions performed in the victim's browser context.

Recommendation:
- Properly validate user input.
- Apply context-aware output encoding.
- Use a strong Content Security Policy.
- Avoid inserting untrusted input directly into HTML or JavaScript contexts.

Evidence:



7.3 SQL Injection

SQL Injection was identified while testing a GET request related to the products functionality.

The parameter supplied to the endpoint was modified during testing and the application's response showed behavior consistent with an SQL injection vulnerability.

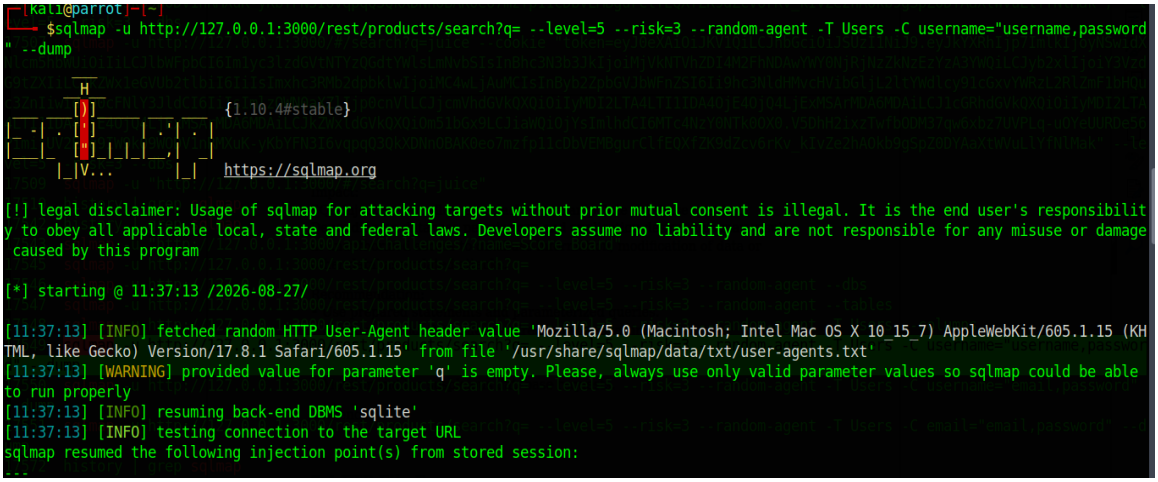
Impact:
SQL Injection can allow an attacker to manipulate database queries. Depending on database

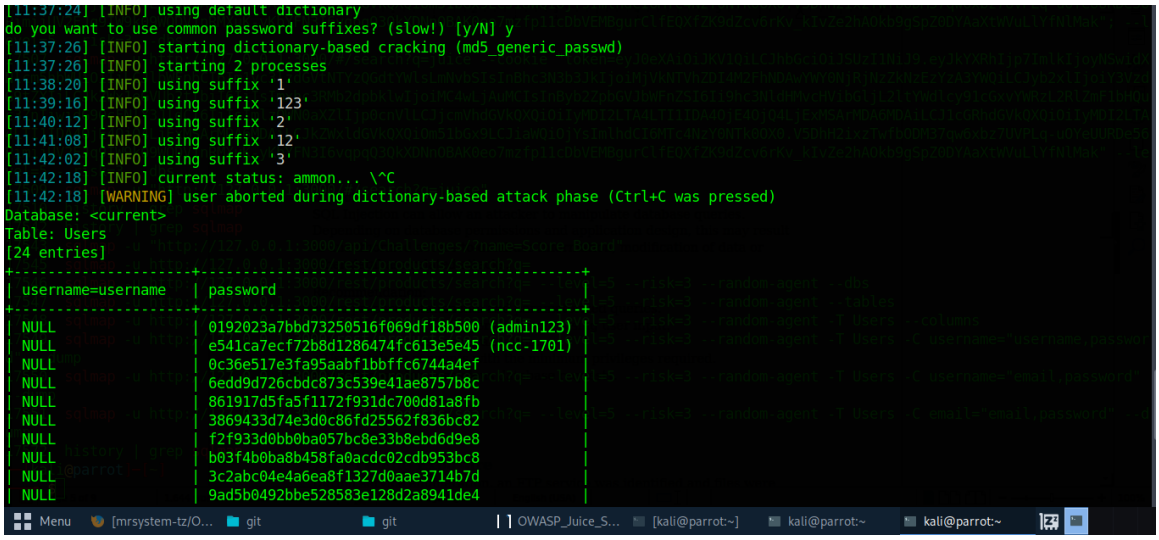
permissions and application design, this may result in unauthorized access to database information, modification of data or other database-related attacks.

Recommendation:

- Use prepared statements and parameterized queries.
- Never construct SQL queries directly using user input.
- Apply server-side input validation.
- Use database accounts with the minimum privileges required.
- Monitor and log suspicious database requests.

Evidence:





7.4

FTP File Exposure

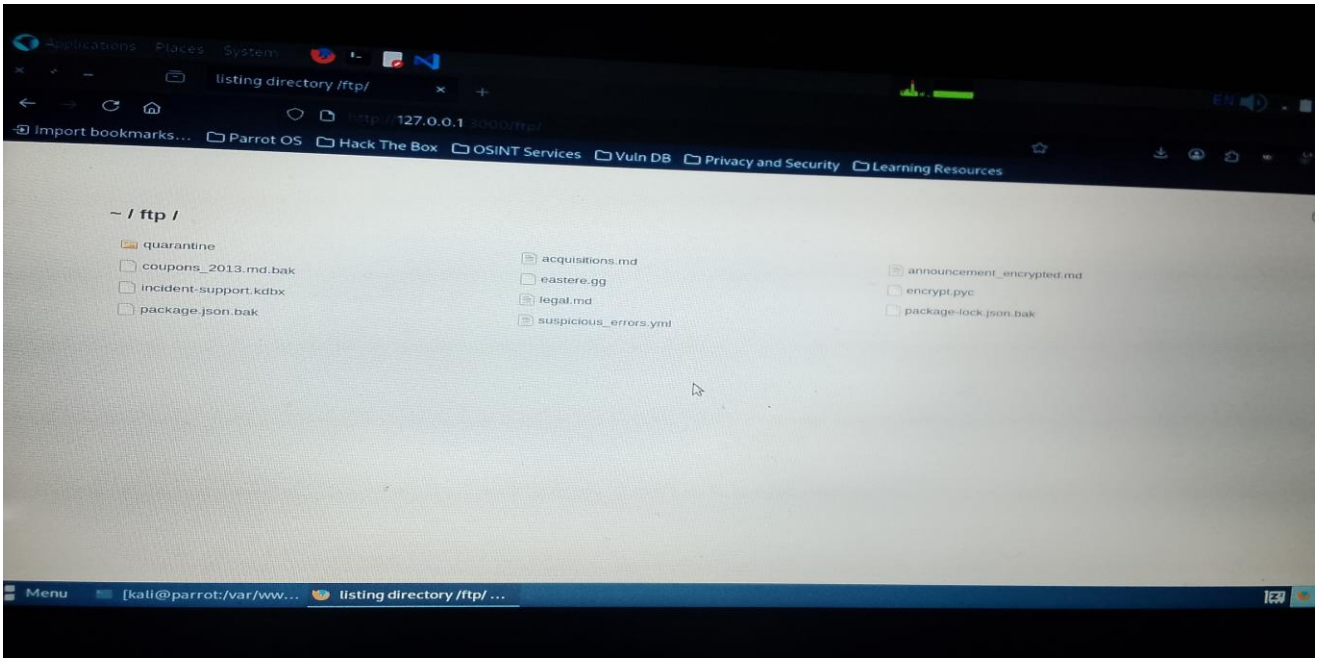
During service enumeration, an FTP service was identified and files were accessible through the service.

The exposure of files through FTP increases the attack surface, especially where access controls are weak or unnecessary files are available.

Impact:
An attacker may obtain files that contain useful information about the application or its environment. If sensitive files are exposed, this could lead to information disclosure and provide additional information for further attacks.

Recommendation:
- Disable FTP if it is not required.
- Restrict access to required users only.
- Remove unnecessary files from publicly accessible locations.
- Use secure file transfer mechanisms where file transfer is necessary.

Evidence:



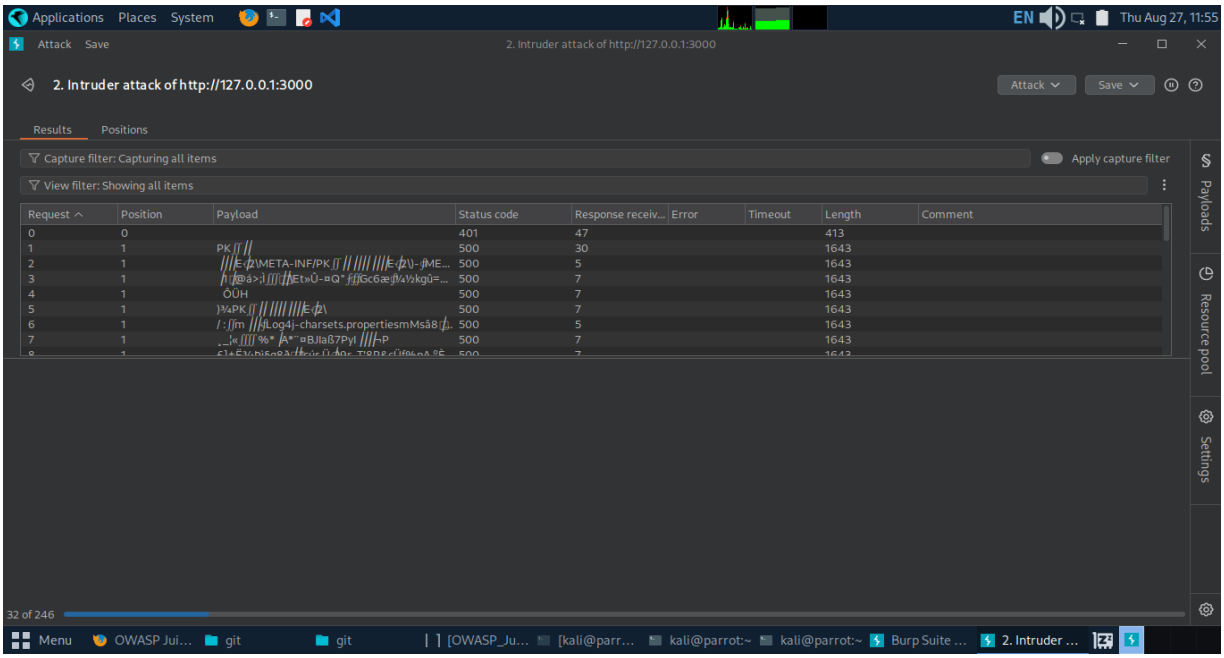
7.5 Security Misconfiguration

Security misconfiguration was observed during the assessment. Some application or server settings were not configured according to a secure baseline.

Impact:
Poor configuration can expose unnecessary services, information or functionality and can make other vulnerabilities easier to exploit.

Recommendation:
- Apply secure server and application configurations.
- Disable unnecessary services and features.
- Remove default or test configurations before deployment.
- Regularly review security settings and apply security updates.

Evidence:



it shows 500 status code response seems the parameter I ijencted their not sanitized in backend and server configuration

7.6 Improper Error Handling / Information Disclosure

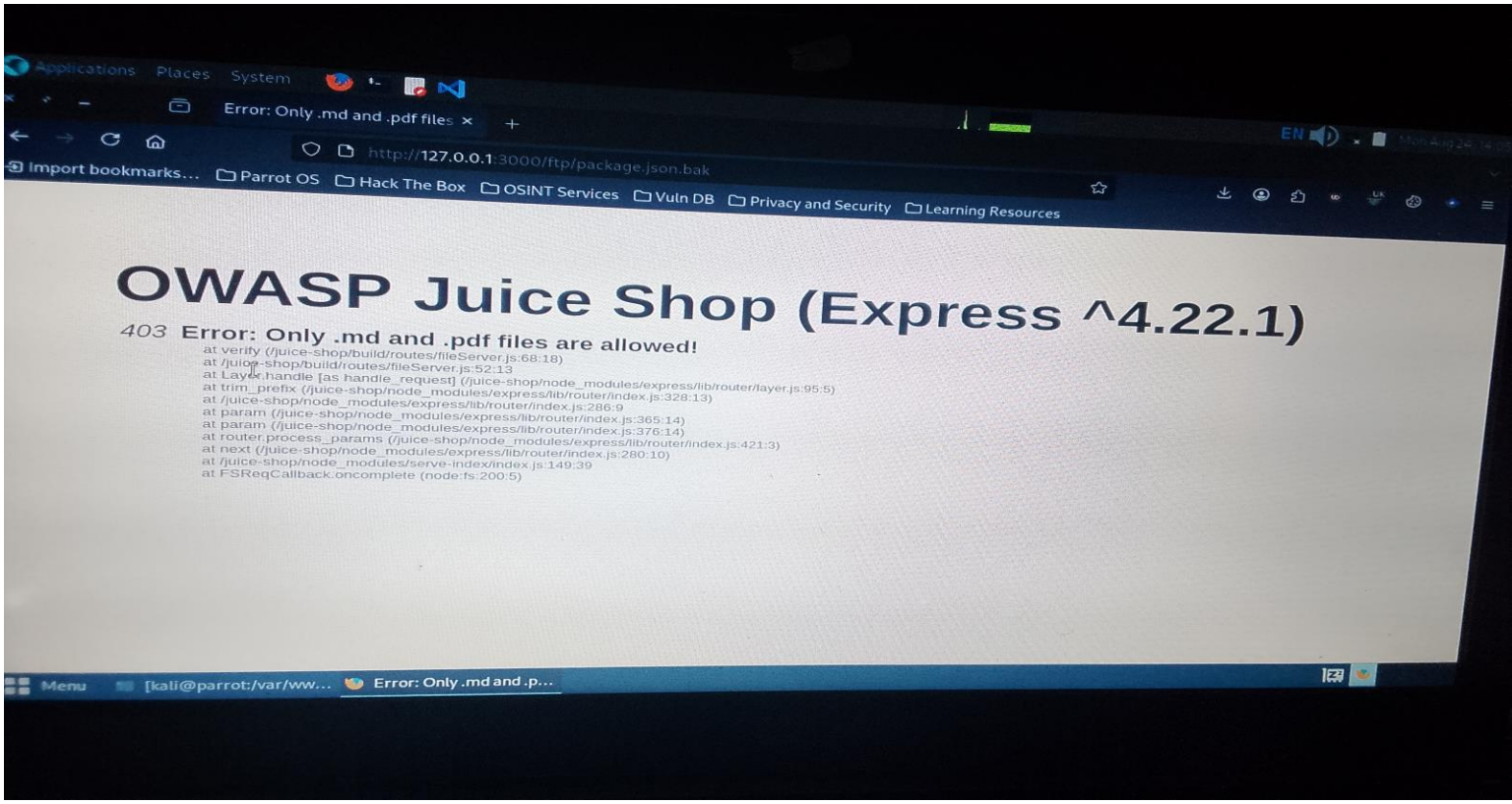
During testing, application errors were observed without adequate handling. The responses could provide information that would normally not be necessary for an ordinary user.

Impact:
Detailed errors can help an attacker understand the application's internal behavior, technologies, endpoints or database-related operations. This information can make further attacks easier.

Recommendation:

- Display generic error messages to users.
- Keep detailed technical information in protected server-side logs.
- Avoid exposing stack traces, database errors or internal configuration details.

Evidence:



7.7 Sensitive Information Exposure

The application was also reviewed for information that may be unnecessarily exposed through API responses, client-side resources or other application components.

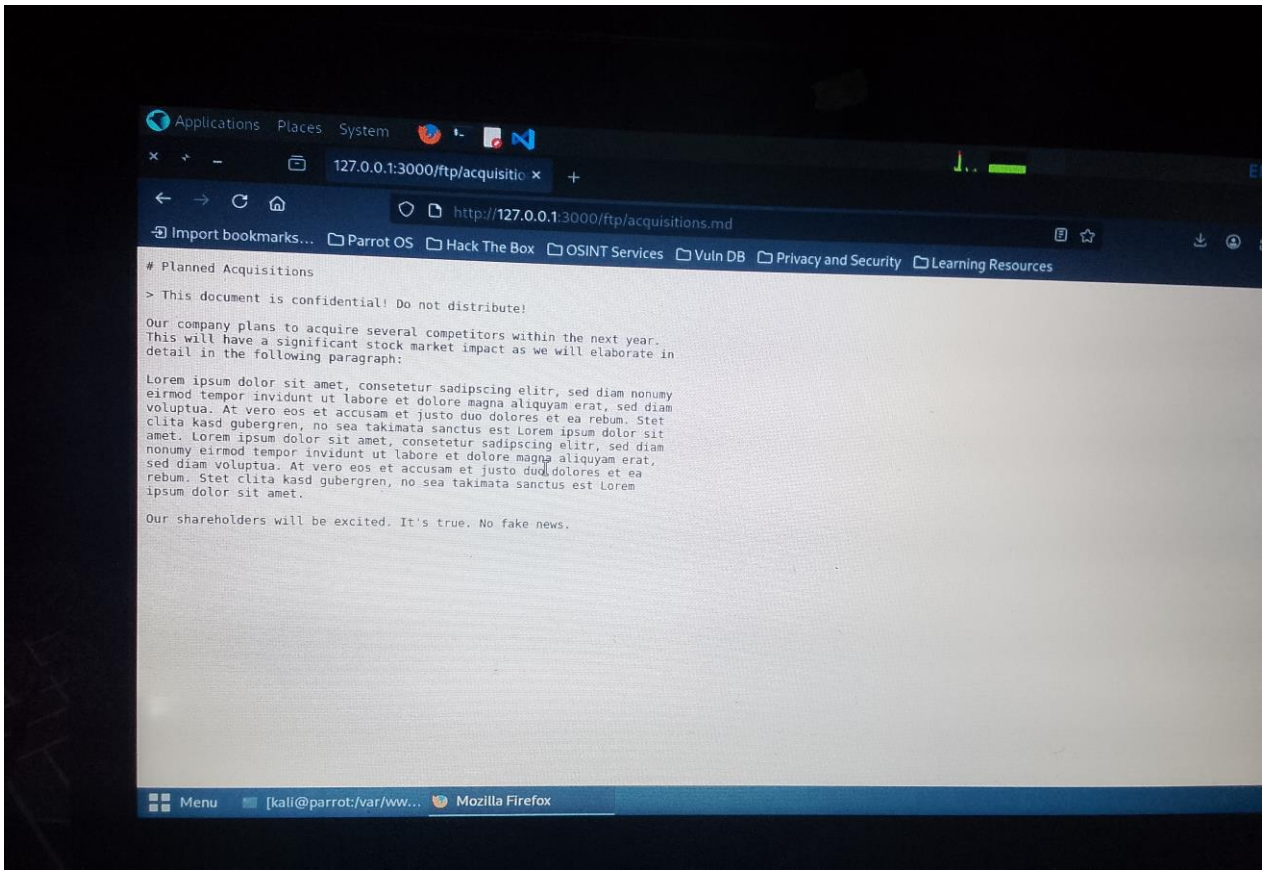
Impact:

Exposed information can provide attackers with useful details about users, application functionality or the underlying environment.

Recommendation:

- Return only the information required by the client.
- Remove sensitive information from client-side code and unnecessary API responses.
- Review logs, configuration files and publicly accessible resources for accidental disclosure.

Evidence:



8. Overall Risk Assessment

The assessment identified several security weaknesses in the OWASP Juice Shop application, including input validation problems, SQL Injection, XSS, exposed files, security misconfiguration and information disclosure.

The most serious findings are SQL Injection and Reflected XSS because they can potentially be

used to affect application data or execute malicious code in a user's browser. Exposed services and files also increase the attack surface of the application.

The findings show the importance of applying secure coding practices, proper input validation, server-side authorization, secure configuration and appropriate error handling.

9. Recommendations

- Use parameterized queries and prepared statements for all database operations.
- Validate and safely encode all user-controlled input before displaying or processing it.
- Implement proper authentication and server-side authorization.
- Restrict or disable unnecessary services such as FTP.
- Remove sensitive and unnecessary files from exposed locations.
- Configure appropriate security headers and secure application settings.
- Use generic error messages while keeping detailed errors in protected logs.
- Minimize sensitive information returned by APIs.
- Perform regular vulnerability assessments and security testing.
- Keep the application, server and dependencies updated.

10. Conclusion

The OWASP Juice Shop assessment demonstrated that a web application can contain several weaknesses across its frontend, backend APIs and supporting services.

The testing successfully identified vulnerabilities including Reflected XSS, SQL Injection, FTP file exposure, exposed API information, security misconfiguration and improper error handling. Other areas such as access control and sensitive information exposure were also reviewed and should be confirmed with specific evidence before being reported as fully exploitable findings.

Addressing these vulnerabilities would reduce the application's attack surface and improve its overall security. The assessment also provided practical experience in identifying vulnerabilities through browser inspection, HTTP request analysis and manual security testing.